

Assumindo: $M = \{ \{a, b, c\}, \{0, 1, 2\}, \delta, 1, \{2\} \}$
 onde:

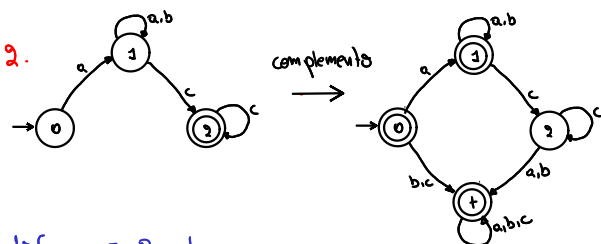
δ	a	b	c
0	1	-	-
1	1	1	2
2	-	-	2

↳ Pela função transição podemos ver que:

$\forall q \in \{1, 2\}, \forall a \in \{a, b, c\}, \exists! p \in \{1, 2\} \mid \delta(q, a) = p$, ou seja, para todo estado e símbolo existe um único estado destino. Sendo assim, o autômato é um AFD.

↳ Expressão Regular: $a(a+b)^*c^+$

↳ Gramática: $G = (\{S, A, B\}, \{a, b, c\}, P, S)$, $P = \{ S \rightarrow aA, A \rightarrow aA \mid bA \mid cB, B \rightarrow cB \mid c \}$



↳ Expressão Regular:

$a(a+b)^*[\epsilon + c^+(a+b)(a+b+c)^*] + (b+c)(a+b+c)^*$

3. a) $L_1 = \{ a^n b^m \mid n \geq 1 \text{ e } m \geq 2 \}$, é regular definida pela Expressão regular $a^+ b a^+$. Gramática Regular $G = (\{S, A, B\}, \{a, b\}, P, S)$,

$P = \{ S \rightarrow aA, A \rightarrow aA \mid b a B, B \rightarrow a B \mid a \}$

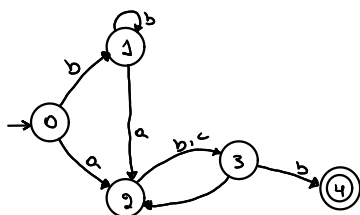
b) $L_2 = \{ a^n b^m a^n \mid n \geq 0 \text{ e } m \geq 1 \}$, Gramática Livre de Contexto

$G = (\{S, A, B\}, \{a, b\}, P, S)$, $P = \{ S \rightarrow aA a \mid b B, A \rightarrow a A a \mid b B, B \rightarrow b B \mid \epsilon \}$

c) $L_3 = \{ a^n b^m a^n \mid n \geq 1 \}$, Gramática Sensível ao Contexto.

d) $L_4 = \{ a^n b^m a^n \mid n \geq 4 \text{ e } n < 100 \}$, Gramática Regular.

4. AFD:



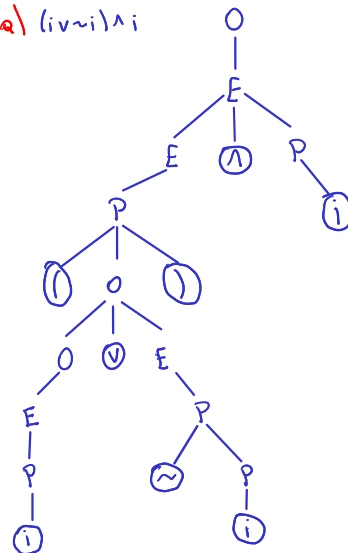
Gramática Regular: $G = (\{S, A, B, C\}, \{a, b\}, P, S)$, $P = \{ S \rightarrow bA \mid b, A \rightarrow bA \mid bB \mid bC \mid b, B \rightarrow aB \mid aC \mid a b b, C \rightarrow aC \mid aC \mid a c b \}$

5. Está no final!

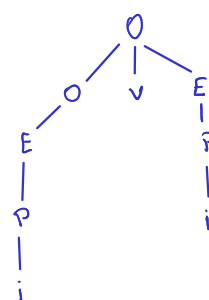
6. $L = \{ 0^n 1^n \mid n \geq 1 \} \cup \{ 0^n 1^{2n} \mid n \geq 1 \}$

$G = (\{S, A, B\}, \{0, 1\}, P, S)$, $P = \{ S \rightarrow 0A1 \mid 0B11, A \rightarrow 0A1 \mid \epsilon, B \rightarrow 0B11 \mid \epsilon \}$

7. a) $(i \vee \sim i) \wedge i$

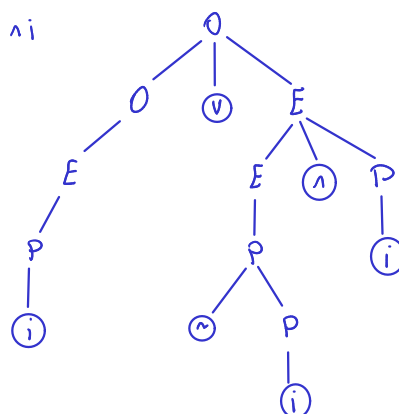


b) $i \sim i$

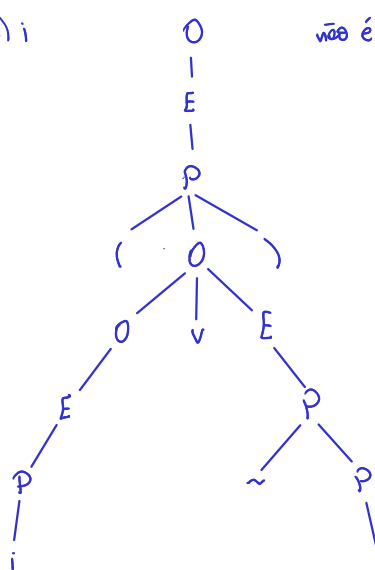


não é possível!

c) $i \vee \sim i$



d) $(i \vee \sim i) \wedge i$



não é possível!

8.

Associatividade:

- $0 \rightarrow 0 \vee E \mid E$: Aqui, 0 pode gerar $0 \vee E$. Isso significa que o operador $\vee$ é aplicado de forma recursiva à esquerda. Então $\vee$ tem associatividade à esquerda.
- $E \rightarrow E \wedge P \mid P$: Similarmente, E gera $E \wedge P$, então o operador $\wedge$ também é associativo à esquerda.
- $P \rightarrow (0) \mid \sim P \mid i$: $\sim$ é um operador unário aplicado diretamente ao P . Isso indica que $\sim$ tem maior precedência, porque é aplicado antes de $\vee$ e $\wedge$.
- Parênteses (0) permitem sobrescrever a ordem de precedência.

• Sendo assim:

- $\vee$ em $0 \rightarrow 0 \vee E \rightarrow$ recursão à esquerda $\rightarrow$ associativo à esquerda
- $\wedge$ em $E \rightarrow E \wedge P \rightarrow$ recursão à esquerda $\rightarrow$ associativo à esquerda

Precedência: Analisando as questões 7a e 7c e sabendo que a precedência é definida pelo nível mais "profundo" da gramática, temos a seguinte hierarquia:

- 1º: $\sim$ (unário) $\rightarrow$ maior precedência
- 2º: $\wedge$ (binário) $\rightarrow$ precedência intermediária
- 3º: $\vee$ (binário) $\rightarrow$ menor precedência

9. $G = \{ \{S\}, \{a\}, P, S \}, P = \{ S \rightarrow SS^+ \mid SS^* \mid a \}$ a) Cadeia $aa+a^*$, usando algoritmo de Early:

D_0 :	$D_1: a$	$D_2: aa$	$D_3: aa+$
$S \rightarrow SS^+/0$	$S \rightarrow a./0$	$S \rightarrow a./1$	$S \rightarrow SS^+./0$
$S \rightarrow SS^*/0$	$S \rightarrow S.S^*/0$	$S \rightarrow SS.^+/0$	$S \rightarrow S.S^*/0$
$S \rightarrow a./0$	$S \rightarrow S.S^*/0$	$S \rightarrow SS.^*/0$	$S \rightarrow S.S^*/0$
	$S \rightarrow SS^+/1$	$S \rightarrow S.S^+/1$	$S \rightarrow SS^+/3$
	$S \rightarrow SS^*/1$	$S \rightarrow S.S^*/1$	$S \rightarrow SS^*/3$
	$S \rightarrow a./1$	$S \rightarrow SS^+/2$	$S \rightarrow a./3$
		$S \rightarrow SS^*/2$	
		$S \rightarrow a./2$	

$D_4: aa+a$	$D_5: aa+a^*$
$S \rightarrow a./3$	$S \rightarrow SS^+./0 \rightarrow$ cadeia aceita, derivação
$S \rightarrow SS.^+/0$	$S \rightarrow S.S^*/0 \Rightarrow SS^* \Rightarrow Sa^* \Rightarrow SS+a^* \Rightarrow Sa+a^* \Rightarrow$
$S \rightarrow SS.^*/0$	$\Rightarrow aa+a^*$
$S \rightarrow S.S^+/3$	$S \rightarrow SS^+/5$
$S \rightarrow S.S^*/3$	$S \rightarrow SS^*/5$
$S \rightarrow SS^+/4$	$S \rightarrow a./5$
$S \rightarrow SS^*/4$	
$S \rightarrow a./4$	

b) $a+a$

D_0 :	$D_1: a$	$D_2: a+$	$D_4: a+a$
$S \rightarrow SS^+/0$	$S \rightarrow a./0$	$\emptyset$	$\emptyset \rightarrow$ a cadeia $a+a$
$S \rightarrow SS^*/0$	$S \rightarrow S.S^*/0$		não pode ser gerada
$S \rightarrow a./0$	$S \rightarrow S.S^*/0$		por G .
	$S \rightarrow SS^+/1$		
	$S \rightarrow SS^*/1$		
	$S \rightarrow a./1$		

10. Uma gramática é ambígua se existe pelo menos uma cadeia na linguagem gerada por ela que possui duas ou mais árvores de derivação distintas, ou seja, duas formas diferentes de derivar a mesma cadeia usando a gramática.

11. $L = \{ a^k b^n c^m \mid n > k+m \}$, $G = (\{S, A, B, C\}, \{a, b, c\}, P, S)$ $P = \{ S \rightarrow ABC,$ $A \rightarrow aAb \mid \epsilon,$ $C \rightarrow bCc \mid \epsilon,$ $B \rightarrow bB \mid b \}$ 12. Assumindo $G = (\{P, A, B\}, \{a, b, c\}, \emptyset, P)$, onde $\emptyset = \{ P \rightarrow AB,$ $A \rightarrow aAb \mid c$ $B \rightarrow bBc \mid a \}$ a) $P \Rightarrow AB \Rightarrow aAbB \Rightarrow acbB \Rightarrow acbbBc \Rightarrow acbbbBcc \Rightarrow \dots \Rightarrow acbbbaacc$ b) $L = \{ a^k c b^{n+m} a^m \mid n \geq 0 \text{ e } m \geq 0 \}$

Ambiguidade em: if b then if b then c else c
 pode ser: 1: if b then (if b then c) else c
 2: if b then (if b then c else c)

Solução:

$$\langle \text{cmd} \rangle \rightarrow \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{cmd} \rangle$$

$$\mid \langle \text{cmd1} \rangle$$

$$\langle \text{cmd1} \rangle \rightarrow \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{cmd1} \rangle \text{ else } \langle \text{cmd} \rangle$$

$$\mid c$$
 $\langle \text{expr} \rangle \rightarrow b$

5.

União:

Se L_1 é uma LR então existe $A_1 = (\Sigma_1, Q_1, \delta_1, q_{01}, F_1)$ que reconhece L_1 .Se L_2 é uma LR então existe $A_2 = (\Sigma_2, Q_2, \delta_2, q_{02}, F_2)$ que reconhece L_2 .Sendo $Q_1 \cap Q_2 = \emptyset$ Podemos construir um AFN com estado inicial q_i , tal que $q_i \notin Q_1 \cup Q_2$: $A_U = (\Sigma_1 \cup \Sigma_2, Q_1 \cup Q_2 \cup \{q_i\}, \delta_U, q_i, F_1 \cup F_2)$.Sendo que δ_U contém todas as transições de δ_1 e δ_2 incluindo as transições $\delta_U(q_i, \epsilon) \rightarrow \{q_{01}\}$ e $\delta_U(q_i, \epsilon) \rightarrow \{q_{02}\}$.

Complemento:

Seja L uma LR, então existe $A = (\Sigma, Q, \delta, q_0, F)$ que reconhece L . Então é possível criar um novo estado $q_i \notin Q$ que receberá as transições que faltam para tornar δ uma função total. Além disso, é possível transformar os estados finais em não finais vice-verso. Desta maneira, obtemos um Automato que reconhece a linguagem complementar a L , ou seja $\bar{L}$.

Interseção:

Se L_1 é uma LR, existe um $A_1 = (\Sigma_1, Q_1, \delta_1, q_{01}, F_1)$ que reconhece L_1 . Se L_2 é uma LR, então existe um $A_2 = (\Sigma_2, Q_2, \delta_2, q_{02}, F_2)$ que reconhece L_2 . Temos que: $L_1 \cap L_2 = \overline{L_1 \cap L_2} = \overline{L_1} \cup \overline{L_2}$. Como foi demonstrado que as LR são fechados para União e Complemento, temos então o fechamento para a interseção.